LUMRA ERP

Page Architecture Blueprint

BAGIAN 2: SALES, POS & INVENTORY ADVANCED

Disusun: 05 May 2026

DAFTAR ISI - BAGIAN 2

1. POS SYSTEM (POINT OF SALE)

1.1 Dashboard POS

File Path: sales/pos_dashboard.html
UI/UX: Quick stats cards: Total Sales Today, Transaction Count, Top Products, Active Sessions. Button besar untuk 'Start New Transaction' & 'View Session'. Multi-location selector. Filter by date range
Backend: GET: Aggregate transactions dari Orders dengan date filter. Calculate metrics real-time dari cache atau DB
Database: LumraConfigOrders (main), OrderLines (for details). Aggregate: COUNT, SUM(total_amount)
Flow: POS Menu -> Dashboard -> Click 'Start New Transaction' -> Go to POS Terminal
Architecture: Cache metrics 10 menit. Use Q objects untuk multi-location filtering

1.2 Halaman Transaksi POS (POS Terminal)

File Path: sales/pos_terminal.html
UI/UX: Split screen: LEFT (Product grid/list dengan search & categories), RIGHT (Cart dengan items, qty, price, subtotal, tax, total). Buttons: Add to Cart (auto), Void Item, Discount, Pay, Cancel Transaction. Customer field. Payment method selector. Real-time calculation
Backend: GET: Load products, categories, tax settings, customer list. AJAX endpoint untuk: Add Item to Cart, Update Qty, Calculate Tax
Database: ProductItems (read: price, available qty), ProductCategories (read: list), TaxSettings (read: tax_rate)
Frontend Tech: Alpine.js atau Vue untuk cart state management. Real-time validation: Check product exists, qty <= available stock
Flow: Search Product -> Click Add -> Auto calculate total -> Select Customer (optional) -> Pay -> Create Order
Special Handling: Cart stored in session/localStorage. Auto-save every 5s. Prevent submit jika ada validation error

1.3 Cart Management (AJAX Endpoint)

File Path: sales/api/pos_cart.py
Endpoint: POST /api/pos/cart/add, /api/pos/cart/update, /api/pos/cart/remove
Logic:
• ADD: POST product_id, qty -> Check stock -> Add to session cart -> Return updated cart JSON
• UPDATE: POST item_id, qty -> Validate qty <= available -> Update session -> Return updated cart
• REMOVE: POST item_id -> Remove dari session -> Return updated cart
• DISCOUNT: POST discount_type (percentage/fixed), amount -> Calculate & apply -> Return total
Response: {success, cart_items[], subtotal, tax, discount, total, message}
Database: ProductItems (check available qty). Query: select_for_update untuk prevent race condition

1.4 Payment & Receipt

File Path: sales/pos_payment.html
UI/UX: Modal atau separate page. Display: Order summary, payment method selector (Cash, Debit, Credit), amount input, change calculation. Print receipt button. Email receipt option
Backend: POST: Create LumraConfigOrders record, OrderLines (per item), InventoryTransaction (update stok), LoyaltyStampTransactions (earn stamps). If payment method = credit -> create AR (Accounts Receivable) record
Database: LumraConfigOrders (write), OrderLines (write), InventoryTransaction (write), LoyaltyStampTransactions (write)
Special Features: Auto-print receipt (WeasyPrint). Email receipt if customer email provided. Warranty tracking
Flow: Cart -> Click Pay -> Select Payment Method -> Enter Amount -> Create Order -> Print Receipt -> Update Loyalty -> Success
Error Handling: Stock suddenly sold out -> Show error & reduce cart qty automatically

2. SALES ORDER & INVOICE

2.1 Daftar Sales Order

File Path: sales/sales_orders_list.html
UI/UX: Tabel: SO No, Customer, Order Date, Total Amount, Status (Draft, Confirmed, Shipped, Delivered, Cancelled). Filter: Status, Date Range. Search: SO No atau Customer. Action: View, Edit, Convert to Invoice, Cancel
Backend: GET: Fetch SalesOrders + prefetch customer, order lines. Filter & search. Pagination
Database: SalesOrders (main), OrderLines (FK), LumraConfigCustomers (FK)
Flow: Sales -> Sales Orders -> List -> Click SO -> View Details / Convert to Invoice / Edit (if Draft) / Cancel

2.2 Form Sales Order (Create/Edit)

File Path: sales/sales_order_form.html
UI/UX: Tabs: Header (SO No, Customer, Order Date, Delivery Date, Terms), Items (Line item tabel dengan Product, Qty, Unit Price, Discount, Total), Summary (Subtotal, Tax, Freight, Grand Total), Notes
Backend: GET (create) form kosong + customer dropdown. GET (edit) pre-fill SO data + line items. POST: Validate customer exists, at least 1 line item, qty > 0. Create SalesOrders + SalesOrderLines. Prevent edit jika status != Draft
Database: SalesOrders (write), SalesOrderLines (write), LumraConfigCustomers (read for dropdown)
Special: Auto-calculate line totals (qty * unit_price - discount). Real-time calculation of grand total. Line item quantity selector dengan available stock display
Flow: Click 'New SO' -> Form (Tabs) -> Add Line Items -> Specify Customer & Dates -> Submit -> Validate -> Create Order -> Show SO Detail

2.3 Daftar Invoice

File Path: sales/invoices_list.html
UI/UX: Tabel: Invoice No, Customer, Invoice Date, Due Date, Amount, Status (Draft, Sent, Paid, Partially Paid, Overdue, Cancelled). Status indicator colors. Filter: Status, Date Range, Overdue. Search: Invoice No atau Customer. Action: View, Edit, Send, Print, Record Payment
Backend: GET: Fetch invoices + calculate payment status (paid_amount vs invoice_total). Calculate overdue days. Prefetch customer & line items
Database: SalesInvoices (main), InvoiceLines (FK), Payments (FK for paid_amount aggregate). LumraConfigCustomers (FK)
Calculation: paid_amount = SUM(amount) FROM Payments WHERE invoice_id = this. status = 'Paid' if paid_amount >= invoice_total, 'Overdue' if due_date < today AND paid_amount < invoice_total
Flow: Sales -> Invoices -> List -> Filter by Status -> Click Invoice -> View / Print / Record Payment / Send

2.4 Detail Invoice & Actions

File Path: sales/invoice_detail.html
UI/UX: Display: Invoice header (No, Date, Customer, Due Date), line items table (Product, Qty, Unit Price, Amount), payment section (Amount due, Payment history tabel). Buttons: Print, Email, Record Payment, Create Credit Note, Cancel Invoice
Backend: GET: Fetch invoice detail, line items, payments. Calculate: amount_due = total - sum(paid amounts)
POST (Record Payment): Create Payments record (amount, date, method). Update invoice status
POST (Create Credit Note): Create CreditNotes record, link ke invoice. Reduce AR.
Database: SalesInvoices (read), InvoiceLines (read), Payments (read/write), CreditNotes (write), GeneralLedger (write for accounting)
Print: Use WeasyPrint untuk render invoice PDF dengan invoice template (header, items, totals, terms)
Flow: Invoice Detail -> Click 'Record Payment' -> Modal (Amount, Method, Date) -> Submit -> Create Payment Record -> Update Status -> Show Success

3. INVENTORY ADVANCED

3.1 Stock Opname Workflow - Overview

Stock Opname adalah proses fisik menghitung stok untuk memastikan database akurat. Workflow terdiri dari 5 halaman utama: Pilih Lokasi -> Input Hitungan -> Daftar Approval -> Detail Approval -> Session History

Database Models:
• StockOpnameSessions: header (session_id, location_id, status, start_date, end_date, created_by)
• StockOpnameLines: detail (session_id, product_id, system_qty, counted_qty, difference, variance_pct, status)
• StockOpnameApprovals: approval flow (session_id, approver_id, status, notes, approved_at)
Status Flow: DRAFT -> IN_PROGRESS -> COUNTING_COMPLETE -> PENDING_APPROVAL -> APPROVED -> POSTED

3.2 Stock Opname - Select Location

File Path: inventory/opname_select_location.html
UI/UX: Lokasi list dengan status indicator (In Progress, Completed, Draft). Tabel: Location Name, Last Opname Date, Status, Last Counted By. Button: 'Start New Session', 'Continue Existing'
Backend: GET: Fetch all locations + last opname session per location. Show only locations where user has permission
Database: LumraConfigLocations, StockOpnameSessions (for last session info)
Flow: Inventory -> Stock Opname -> Select Location -> Click 'Start New' -> Create new session -> Go to Input Hitungan

3.3 Stock Opname - Input Hitungan

File Path: inventory/opname_input_counting.html
UI/UX: Tabel dengan columns: SKU, Product Name, System Qty, Counted Qty (input field), Difference, Variance %. Products bisa filtered/searched. Counted Qty field auto-calculate difference & variance. Progress bar showing completion %
Backend: GET: Fetch products dalam location, system_qty dari InventoryTransaction. Create empty StockOpnameLines
• AJAX POST untuk setiap counted_qty input: Calculate difference, validate (not negative)
• Prevent counting qty > (system_qty * 150%) to catch input errors
Database: StockOpnameLines (write: counted_qty, difference, variance_pct). InventoryTransaction (read: system_qty)
Special: Drag-drop reorder items jika perlu. Mark item 'Done' saat selesai count. Session auto-save every 2 minutes. Variance report untuk items dengan >10% difference
Flow: Input counting -> Auto-calculate variance -> Mark items as done -> Click 'Complete Counting' -> Go to Approval

3.4 Stock Opname - Approval List & Detail

File Path: inventory/opname_approval_list.html, opname_approval_detail.html
Approval List UI: Tabel: Session ID, Location, Counted By, Count Date, Status (Pending, Approved, Rejected), Items with Variance. Filter: Status, Location. Action: View Detail, Approve, Reject
Detail Page UI: Summary: Location, Date, Counted By. Tab: Items (tabel dengan SKU, Product, System Qty, Counted Qty, Variance). Tab: Variance Report (items >10% diff dengan red highlight). Approval section: Approver notes, Approve/Reject buttons
Backend - Approve: Validation: At least 1 item counted. Calculate & verify variance < threshold (default 5%). Create InventoryTransaction untuk adjust stok (counted_qty - system_qty). Update session status to POSTED. Audit log
Backend - Reject: Reset session to IN_PROGRESS. Add note ke approvals table. Notify counter untuk re-count
Database: StockOpnameSessions (write: status), StockOpnameApprovals (write), InventoryTransaction (write for adjustments)
Flow: Approval List -> Click session -> Review items & variance -> Approve -> Create adjustment transactions -> Update stok -> Success notification

3.5 Stock Opname - Session History

File Path: inventory/opname_session_history.html
UI/UX: Timeline atau tabel history semua opname sessions dengan: Date, Location, Item Count, Variance Count, Approver, Status. Link ke detailed report untuk setiap session
Backend: GET: Fetch all completed StockOpnameSessions ordered by date DESC. Show summary metrics
Database: StockOpnameSessions (read)
Export: CSV export untuk audit trail
Flow: Inventory -> Stock Opname -> Session History -> Click session -> View full opname report

4. STOCK TRANSFER BETWEEN LOCATIONS

4.1 Stock Transfer Form & Workflow

File Path: inventory/stock_transfer_form.html
UI/UX: Form dengan: From Location (dropdown), To Location (dropdown), Items (tabel: Product, Qty, Status). Add items button. Submit creates transfer order
Backend: GET (form): Load locations dropdown. POST: Validate from != to location, items qty > 0 & <= available qty in from location. Create StockTransfers record + StockTransferLines. Create InventoryTransaction 2x: OUTGOING (from location), INCOMING (to location)
Database: StockTransfers (write), StockTransferLines (write), InventoryTransaction (write: 2 records per transfer)
Flow: Inventory -> Stock Transfer -> Select Locations -> Add Items -> Submit -> Create Transfer -> Stock auto-updated
Special: Real-time balance update. Prevent negative stock. Optional approval step jika amount > threshold

5. LOYALTY & CUSTOMER ENGAGEMENT

5.1 Loyalty Dashboard

File Path: sales/loyalty_dashboard.html
UI/UX: KPI cards: Total Members, Active Members, Total Stamps Issued, Total Rewards Redeemed. Tier distribution chart. Reward popularity. Recent redemptions list
Backend: GET: Aggregate dari LoyaltyStampCards, LoyaltyTiers, LoyaltyRedemptions
Database: LoyaltyStampCards (count members), LoyaltyTiers (tier distribution), LoyaltyRedemptions (count rewards)
Flow: Sales -> Loyalty -> Dashboard -> View metrics

5.2 Stamp Card Management

File Path: sales/loyalty_stamp_cards_list.html
UI/UX: Tabel: Card Number, Customer, Tier, Current Stamps, Lifetime Stamps, Last Visit, Status. Filter: Tier, Status (Active/Inactive). Search: Card No atau Customer. Action: View Card, Manage Stamps, Deactivate
Backend: GET: Fetch cards + prefetch customer, tier. Calculate stamps progress to next reward
Database: LoyaltyStampCards (main), LoyaltyTiers (FK), LumraConfigCustomers (FK)
Manual Stamps: POST endpoint untuk manually add/deduct stamps (admin only) dengan reason
Flow: Loyalty -> Stamp Cards -> List -> Click Card -> View Customer Details / Manual Adjust / View Transactions

5.3 Redemption History & Rewards

File Path: sales/loyalty_redemptions_list.html, loyalty_rewards_list.html
Redemptions List UI: Tabel: Card No, Customer, Reward, Stamps Used, Date, Status. Filter: Status, Date Range. Action: View, Cancel Redemption
Rewards List UI: Tabel: Code, Name, Type (Free Item, Discount, Merchandise), Required Stamps, Valid Days, Redeemed Count, Status. Action: View, Edit, Activate/Deactivate
Backend: GET redemptions: Fetch dari LoyaltyRedemptions + prefetch card, reward, order. GET rewards: Fetch rewards + count redeemed per reward
Database: LoyaltyRedemptions (read/write), LoyaltyRewards (read/write), LoyaltyStampCards (read)
Flow: Loyalty -> Redemptions -> View history | Loyalty -> Rewards -> Manage reward catalog

6. LAMPIRAN & BEST PRACTICES

6.1 POS Workflow - Detailed Flow Diagram

START -> Dashboard (stats) -> New Transaction
-> Product Selection (search/category) -> Add to Cart (AJAX)
-> Review Cart (edit qty, remove items) -> Discount Application (optional)
-> Customer Selection (optional) -> Payment Selection (Cash/Card/Other)
-> Create Order (save to DB) -> InventoryTransaction (deduct stok)
-> LoyaltyStampTransactions (earn stamps jika customer present) -> Print Receipt
-> Update Dashboard Stats -> Transaction Complete

ERROR HANDLING:
• Stock sold out during checkout -> Show warning, auto-remove from cart
• Product price changed -> Alert user, ask confirmation
• Discount invalid -> Show error, block payment
• Payment failed -> Rollback transaction, show error message
• Network disconnect -> Save to localStorage, retry on reconnect

6.2 Stock Opname Approval Workflow

START -> Select Location
-> Create Session (DRAFT) -> Input Counting (status: IN_PROGRESS)
-> Calculate Variance -> Mark Complete (COUNTING_COMPLETE)
-> Send to Approver (PENDING_APPROVAL) -> Review Variance Report
-> Approver Decision: APPROVE or REJECT
IF APPROVE:
-> Validate variance < threshold (5%)
-> Create InventoryAdjustment (counted_qty - system_qty)
-> Create InventoryTransaction records
-> Update ProductItems.available_qty
-> Mark Session POSTED
ELSE (REJECT):
-> Reset to IN_PROGRESS
-> Notify counter untuk re-count
END

6.3 Performance Optimization Tips

• POS Terminal: Use select_for_update() saat update inventory untuk prevent race condition
• Cache: Cache product list & categories 10 min. Invalidate saat product update
• Query: Always prefetch_related & select_related untuk avoid N+1 queries
• Cart: Store di session/localStorage untuk fast access tanpa DB hit every time
• Pagination: Max 50 items per page untuk list views. Use cursor pagination untuk large datasets
• Stok calculation: Don't recalculate setiap kali. Cache & update incrementally
• Loyalty: Cache member tier status 5 min untuk fast lookup saat add to cart
• Reporting: Use database views atau pre-calculated tables (materialized views) untuk complex reports